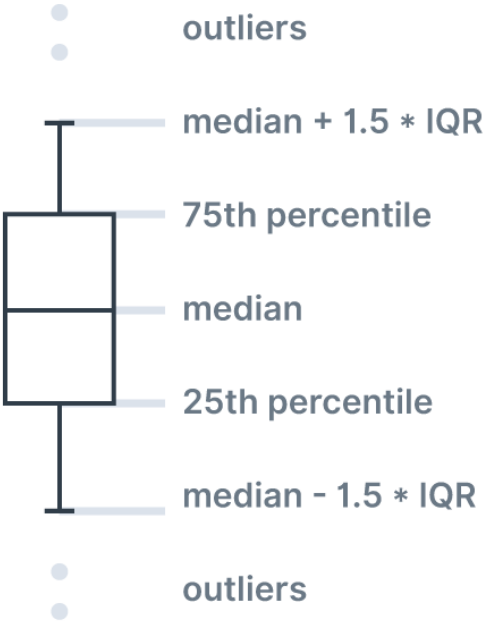
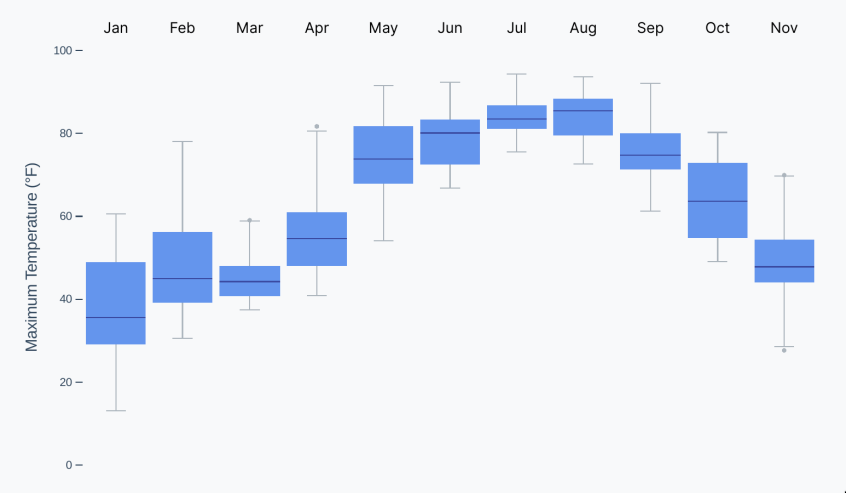


box plot diagram

> Note that there are different conventions for where to place the ends of the whiskers. For example, some charts cover all data points with the whiskers, whereas some charts cover one standard deviation above and below the mean. Our definition of **1.5 times the IQR from the median** is a good middle-ground that will show the variation, but will also be resistant to outliers.

Now that we understand the different parts, let's take another look at our weather box plot.



box plot

By looking for outlier dots, we can see that both March and April had two abnormally hot days. Looking at the whiskers, we can see that there is no overlap between max temperatures in March and June. Additionally, the maximum temperatures in March are all very similar, whereas February has a lot of variability.

If we were just looking at the mean or median temperature of each month, we wouldn't be able to make any conclusions about the range of values. A box plot has added complexity and can take more practice to read, but packs a lot of information about variability.

> Box plots are more complex than the charts we've created before — be sure to check out the code in the `/code/08-common-charts/box-plot/` folder. Note that we create a `<g>` element for each of the months' elements to keep them together and make things easier with our data binding enter/exit pattern.

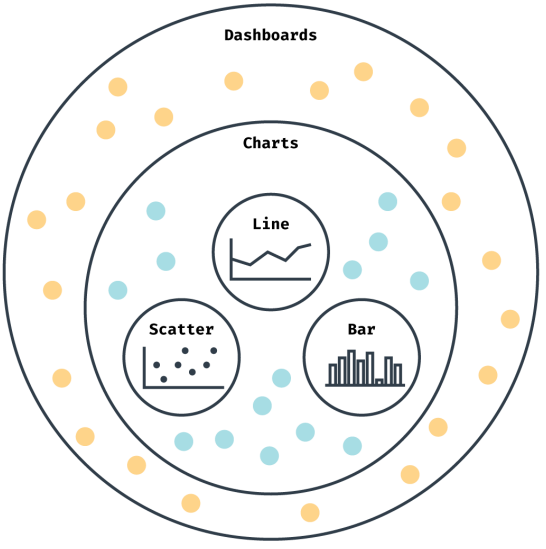
Conclusion

This is just the tip of the iceberg. There are many more types of charts and many chart types that have yet to be discovered! Remember to use this chapter as a starting off

point, or as a source of inspiration. And make sure to look through the code for each example or try to recreate the chart on your own to solidify your skills.

Dashboard Design

So far, we've talked about visualizing data at the **chart** level. Let's zoom out another level and talk about how to design effective **dashboards**.



conceptual scopes

What is a dashboard?

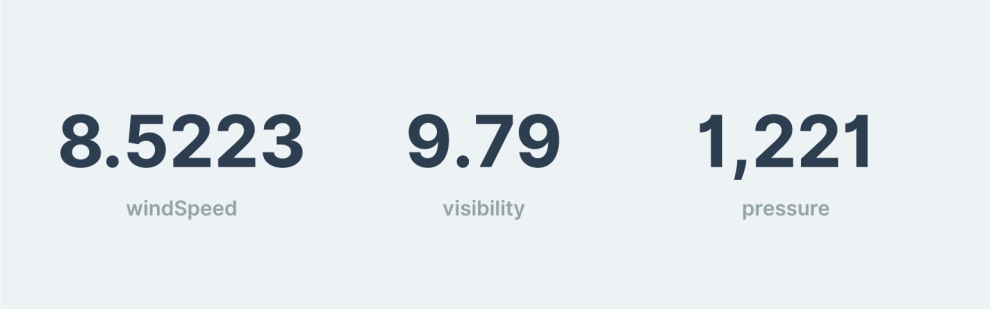
Good question! There are many definitions of "dashboard" - in this chapter, I'll be using the word to refer to any web interface that makes sense out of dynamic data. This is by no means the official definition of **dashboard**, but it is a handy definition for our use here.

We'll talk about ways to display a simple metric effectively, how to deal with different data states when loading data, how to design tables, and then we'll run through a

design exercise. You'll come away with tangible strategies for displaying data in an actionable manner. Let's dig in!

Showing a simple metric

Let's say we have an overview screen and we want to display the three most important metrics. The straightforward solution would be to show numbers with the name of the metric, right? Maybe we'll format the number nicely and call it a day.



metrics

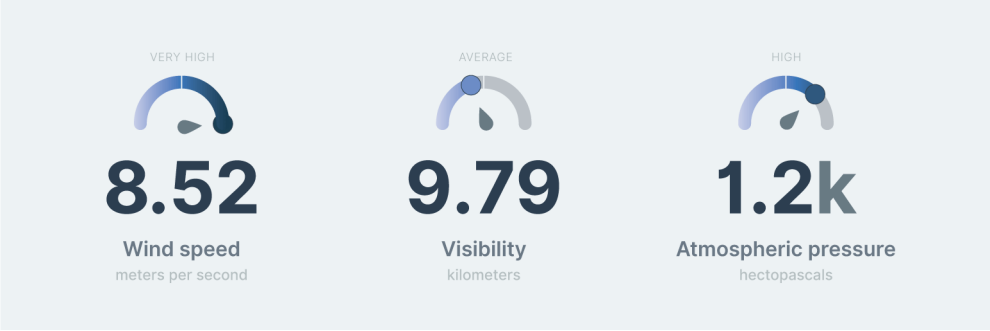
This certainly works! Your users can read the numbers and go from there. But what does "go from there" mean? Let's try to anticipate some questions that need to be answered in order to convert these numbers into insights.

- What units are these numbers in?
- Is 8.52 meters per second fast? How does this number compare to other values?

If you take anything away from this chapter, let it be that **context is king**. A number is meaningless if we don't know:

1. What it means — are we looking at miles per hour or meters per second?
2. How it compares — is this an average value, or is it abnormally high?

Here is a more useful view of our metrics:



metrics finished

We've added a gauge on top of our numbers — the range could be either:

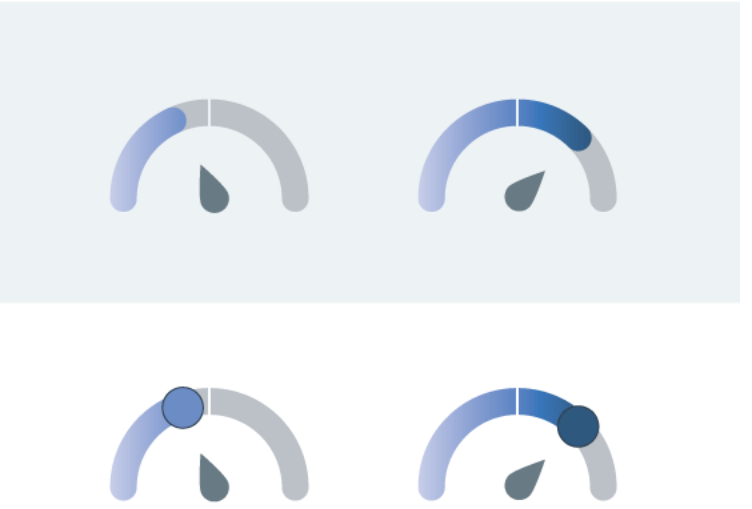
- the total range of possible values, or
- centered around the mean or median and extending two standard deviations in either direction.

The gauge lets the user know how the value compares to other values. Almost as important, though, is letting the user "read" the metric more quickly.

Both the position and the color of the bubble give a fast impression of how the metric compares. A lighter or more left-ward bubble means a low value — that's easy to scan for.

Adding gauges also makes metric comparisons easier — the **atmospheric pressure** is high, but not as abnormally high as the **wind speed**.

But why did we add bubbles to the gauges? Let's take a look at the gauges with and without a bubble. Notice how quickly you can scan both sets.

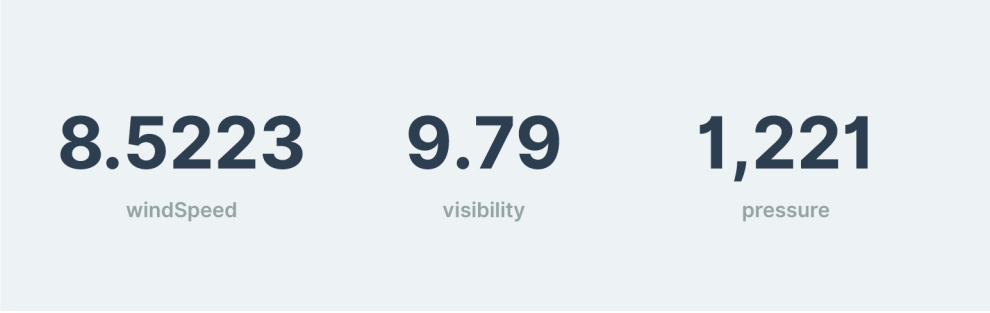


gauges with and without bubbles

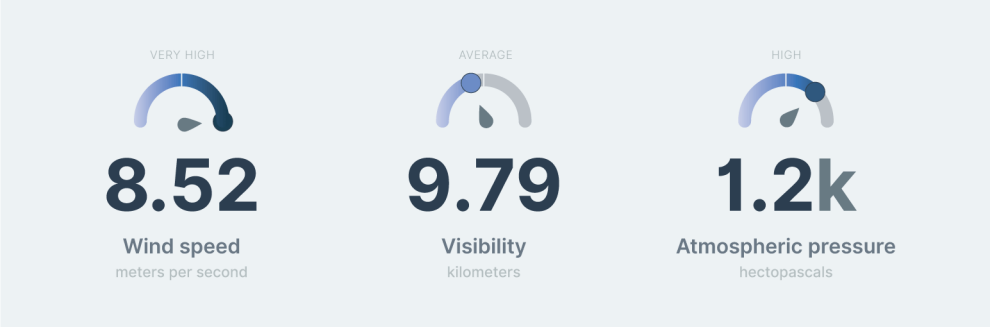
With the background gradients, we're a little overwhelmed with colors — it's hard to isolate the color of the actual value. Sometimes adding an extra, redundant element helps with focusing the user on what matters.

> There are many types of gauges and visual indicators that can help with quickly interpreting numbers. This gauge is a simple example — put into terms we learned in **Chapter 7**, we're representing the metric value with both a **position** and **color** dimension. Think of some other ways we could represent such a value, and what the pros and cons would be of each choice.

Let's look at the other changes we made to make our metrics easier to process:



metrics



metrics finished

We added categorical labels

For example: **very high**, **average**, etc.

Adding descriptive language primes your user with the correct language to use. Sometimes users aren't comfortable making their own conclusions from the data, and turning a chart into words can go a long way towards giving them confidence.

We formatted the numbers to provide just enough information

Figure out what specificity your users will need (eg. two decimal places) and don't show more than that. If necessary, you can add some kind of progressive disclosure, such as showing the full value in a tooltip or in an export.

We dimmed less important information

Decorators such as `%` or `$` can be dimmed since they're not as important as the numbers themselves. This is especially true when comparing metrics - `3` is the important part when comparing `$3` to `$2`.

We added units

For example, **meters per second**, **kilometers**, etc.

Even when it seems obvious what a number represents, more contexts is better. Having units everywhere you display a number also helps keep your dashboard shareable — ask yourself: "Would a screenshot be self-explanatory?"

We added human-friendly labels

It's a good idea to centralize a mapping of metric **keys** (to access the numbers) and your **labels**. Any person would rather read **Wind Speed** than **windSpeed**.

Another perk of a centralized mapping is that your metric names will be consistent throughout your dashboard.

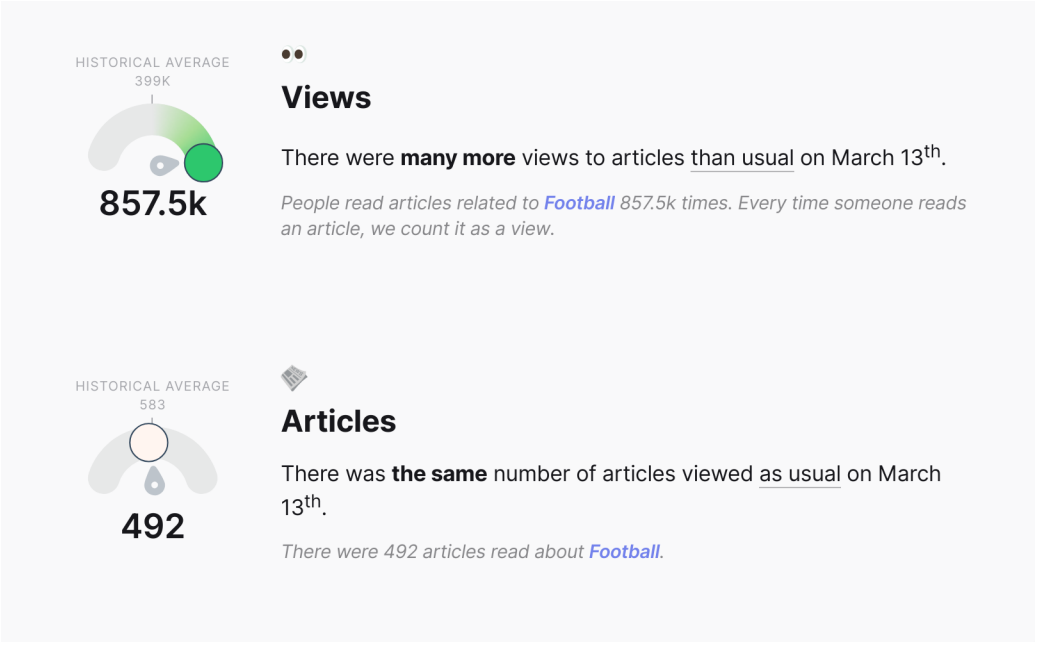
When adding a metric to your dashboard, consider all the ways you can provide context. Would it help to show:

- what the past values have been? Showing where this value lies in a distribution could give the user lots of information.
- what the value was one week ago?
- what the value is in another location?
- what the value is likely to be in the future?

Another way to make simple numbers more insightful is to combine them into a new metric. For example, I was recently working on a dashboard that showed two things:

1. how many times people read an article about a topic, and
2. how many articles were read about that topic.

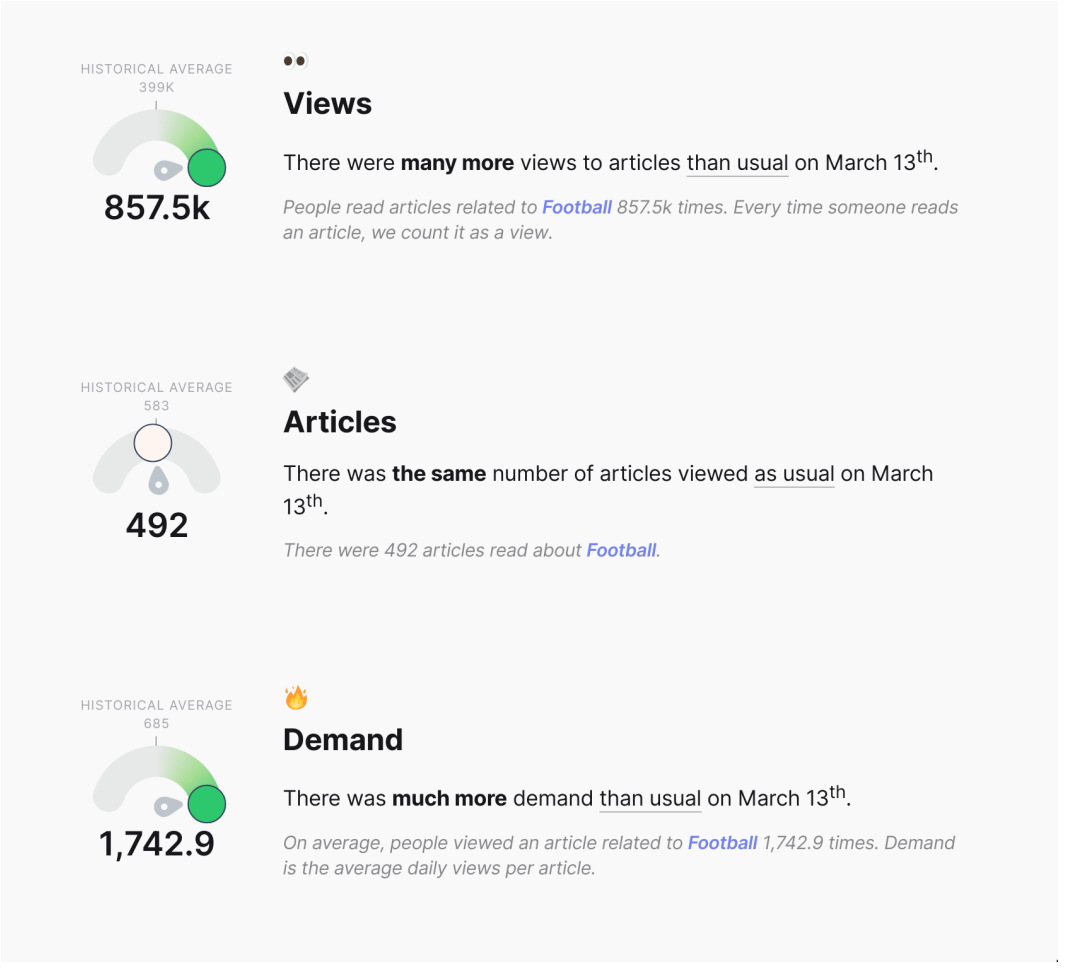
Here they are for the topic **Football**.



views and articles viewed, from the Parse.ly Currents dashboard

Even with the historical context, these values are not very actionable. You might think "Wow, a lot of people are reading about football", then move on with your day.

In order to change **numbers** into **insight**, we created a new metric — **views per article**.



views, articles viewed, and views per article, from the Parse.ly Currents dashboard

Now, a journalist could see this and think "Wow, articles about football are getting a lot of attention (relative to other topics), they must be high in demand!" That's actionable — the journalist can take that information and write that football article they've always wanted to write, knowing that it will interest more people than another topic.

Dealing with dynamic data

Building a chart with static data is a great feat — you need to wrangle your data, position all of the elements, and create an effective design. When creating a dashboard, the amount of work at least doubles. Let's talk about some concerns that come up when creating dynamic charts and how to solve them.

Data states

In addition to the final design, our dashboard charts will need to handle multiple states:

- loading
- loaded
- error
- empty

But what should our charts look like during each of these states?

Loading

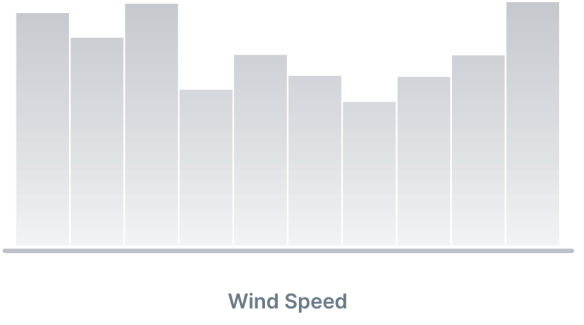
The simplest solution for a loading chart is to display `Loading...` or a loading indicator.

Loading...

loading state (minimal)

While this is better than nothing, we can give our users more information. In the case of data that takes more than a few seconds to load, we should help users decide whether they want to wait, and prepare them for when the data are available.

What extra information can we give our users without distracting them from anything else on the page? A greyed-out version of your final chart often works well.



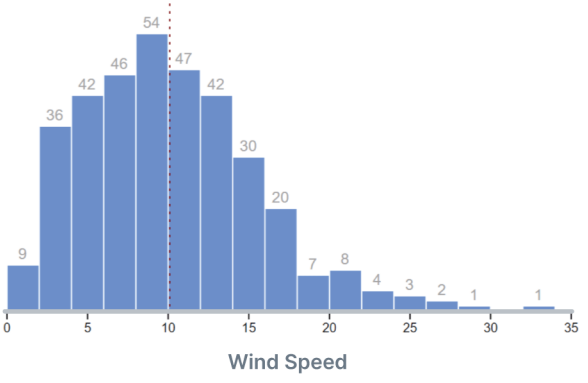
loading state

Additionally, an animation can be helpful to indicate that this state is not final.

> Another solution is to render the chart with empty or random data. This is generally a bad idea, though, since it too closely mimics the loaded state and could confuse the user.

Loaded

When your data have loaded, animating the final state in looks great and also alerts your user that the wait is over!

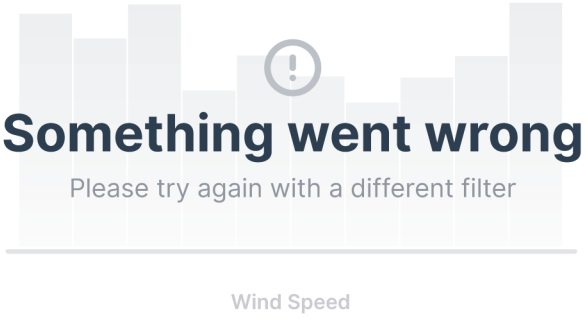


loaded state

Make sure that your loading and loaded states are the same size, so your content isn't jumping around on the page. This is especially important if the user could be looking at something furthur down on the page.

Error

Error states are important signals that something is wrong and the chart won't be appearing. Make sure these are visually distinct enough from your loading state that a waiting user is alerted quickly.

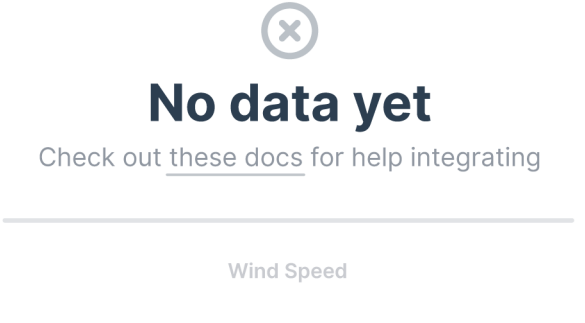


error state

Error states can be jarring — we've all had the experience of waiting for a page to load, just to be rejected. Make sure to tell your users what they can to do to fix the issue: try again later, upgrade their account, change the filter, etc.

Empty

Empty states are a unique error state — instead of signalling that there was an issue, they signal that the data does not exist.



empty state

Empty states require an extra check to see if there are any data values. For example, we might receive an array of empty values for a timeline — in this case, we'll have to run through the whole array to check whether or not we have any non-zero values.

Dancing numbers

If we're showing numbers on our dashboard, it's a good idea to use a monospace font — especially when those numbers will be changing when the data updates or filters change. This will prevent layout changes when the numbers change, plus numbers in a column will be easier to compare (such as in a table).

This might require bringing in a new font, although some fonts have a fixed-width setting that can be toggled via CSS. For example, the font we use in our chart code, `Inter`, can be switched to fixed width with the following CSS.

```
font-feature-settings: 'tnum' 1;
```

normal versus fixed-with numbers

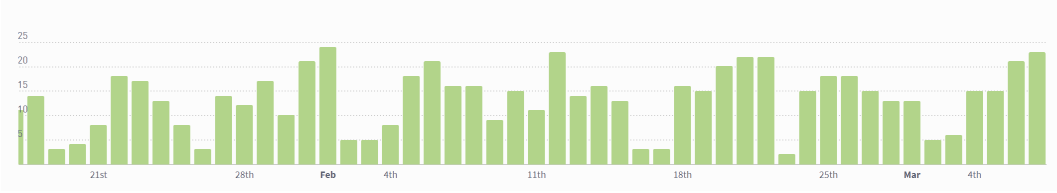
Dealing with outliers

The data we use for our chart can vary wildly — the values might cluster together tightly or we might have to display very different values. When there are extreme outlier values in our data, the scale can be exaggerated and drown out smaller values.

As usual, the solution will depend on the goal of the chart. If the goal is to show a birds-eye-view of the data, letting the outliers skew the whole chart might be ideal.

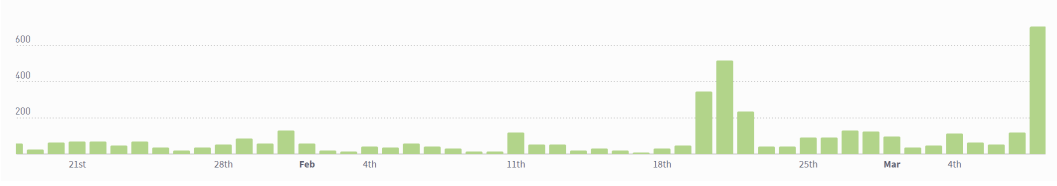
In most cases, though, the goal is to see the pattern for most of the data (and be notified that outliers exist). Let's look at a few examples.

This timeline (in the Parse.ly Analytics dashboard) is designed to show daily views over time for a specific website.



timeline, from the Parse.ly Analytics dashboard

Setting the y scale domain to `[0, max daily views]` usually works, but if the website publishes an article that goes viral, the site might receive four times the typical daily traffic. While exciting, this will blow out the y scale and make daily trends hard to see.



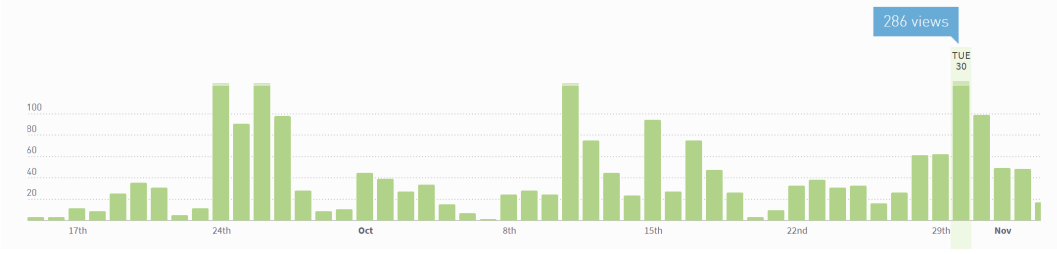
timeline with outliers, from the Parse.ly Analytics dashboard

To make the solution clear, let's lay out the goal for this chart: to show website owners how much traffic their site is getting — is that number increasing or decreasing over time? Are there weekly or montly patterns? Are there any recent changes that they need to be aware of?

With this in mind, we'll apply two rules:

1. If an outlier exists in recent data (say, the past week), we'll use the default y scale domain of `[0, max daily views]`. This enables viewers to see the extremity of an ongoing traffic spike — something to celebrate!

2. If an outlier exists more than 7 days ago, it will get cropped and we'll define our y scale domain as `[0, mean daily views + 2.5 standard deviations]`. We don't want to lose this data, though! We'll show a stripe on top of the day's bar, with extra stripes for more extreme outliers. This way we essentially have two y scales — one for most of the bars and one for the outlier stripes, enabling us to see both the general pattern and get a sense of extreme traffic patterns.



timeline with cropped outliers - cropped, from the Parse.ly Analytics dashboard

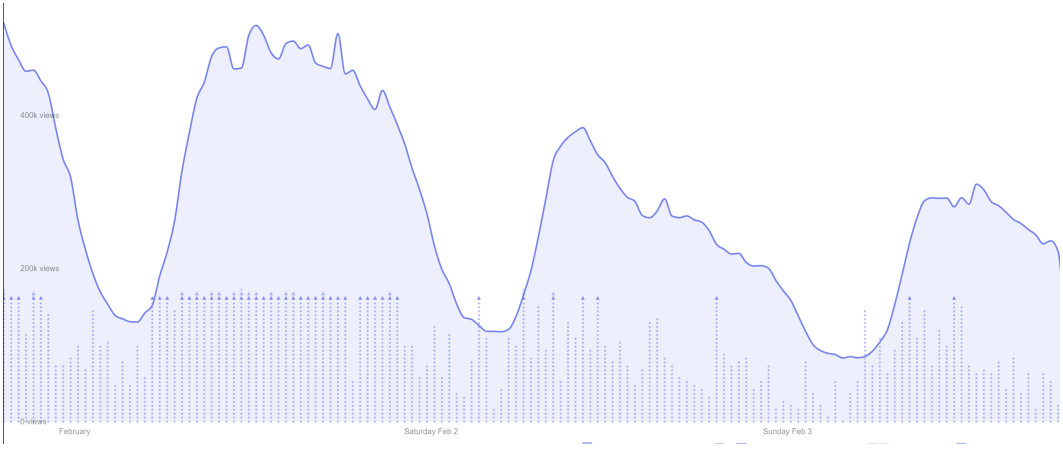
This pattern is mimicked in Parse.ly's other dashboard (Parse.ly Currents). The following chart shows traffic over time (the line/area) *and* content production (the dots).

timeline, from the Parse.ly Currents dashboard

In an effort to prevent overwhelming the main chart with the enhancement of content production data, the dot scale is restricted to have one dot represent one article. We also want to keep the dots from getting too tall, so we'll cap the dot's y

scale at 1/3 the height.

Unfortunately, some data have more articles than we have room to display. The solution here is similar to the previous solution — have another, abbreviated, y scale for outliers. In this case, arrows are used instead of stripes, since they are similar to the dots but different enough to stand out.



timeline with cropped outliers, from the Parse.ly Currents dashboard

There are many creative ways to show outliers while preserving the rest of the chart, but how do we detect outliers?

There are various definitions of an "outlier". A good general-use definition is any value that is more than 2.5 standard deviations from the mean.

Designing tables

Often we'll want to show the exact values in our dataset. In this case, it's hard to beat a simple table. Although designing a table sounds straightforward, there are ways to make a simple table more engaging and easier to read.

Let's redesign a table of our weather data — feel free to follow along at

`/code/09-dashboard-design/table/draft/`.

Writing the code along with these changes is a great way to practice your d3 and CSS skills. If you get stumped, check out the finished code at

`/code/09-dashboard-design/table/completed/`

Even if you're not following along, check out the original table at http://localhost:8080/09-dashboard-design/table/draft/[63].

We'll start out with an unstyled table showing several metrics (columns) over multiple days (rows).

Day	Summary	Max Temp	Max Temp Time	Wind Speed	Precipitation	UV Index
2017-01-03	Clear throughout the day.	45.34	1483423200	7.63	rain	1
2017-01-04	Foggy in the evening.	46.96	1483509600	8.92	rain	2
2017-01-05	Mostly cloudy until evening.	43.53	1483628400	13.42	rain	2
2017-01-06	Mostly cloudy starting in the afternoon, continuing until evening.	47.13	1483682400	3.4	rain	1
2017-01-07	Overcast until evening.	45.5	1483736400	6.7	rain	2
2017-01-08	Overcast until evening.	42.47	1483812000	13.07	rain	2
2017-01-09	Partly cloudy until afternoon.	39.22	1483909200	14.66	rain	2
2017-01-10	Mostly cloudy until evening.	35.61	1484006400	11.76	snow	2
2017-01-11	Partly cloudy in the morning.	38.83	1484125200	6.46	rain	2
2017-01-12	Mostly cloudy until evening.	37.95	1484168400	6.19	rain	2

Our table

This is pretty hard to read, with cramped cells and dark borders. To start, we'll add padding to each cell and take away the borders.

Day	Summary	Max Temp	Max Temp Time	Wind Speed	Precipitation	UV Index
2017-01-03	Clear throughout the day.	45.34	1483423200	7.63	rain	1
2017-01-04	Foggy in the evening.	46.96	1483509600	8.92	rain	2
2017-01-05	Mostly cloudy until evening.	43.53	1483628400	13.42	rain	2
2017-01-06	Mostly cloudy starting in the afternoon, continuing until evening.	47.13	1483682400	3.4	rain	1
2017-01-07	Overcast until evening.	45.5	1483736400	6.7	rain	2
2017-01-08	Overcast until evening.	42.47	1483812000	13.07	rain	2
2017-01-09	Partly cloudy until afternoon.	39.22	1483909200	14.66	rain	2
2017-01-10	Mostly cloudy until evening.	35.61	1484006400	11.76	snow	2
2017-01-11	Partly cloudy in the morning.	38.83	1484125200	6.46	rain	2
2017-01-12	Mostly cloudy until evening.	37.95	1484168400	6.19	rain	2

Our table, padded

Unfortunately, without those borders it's hard to separate each row. We could add the borders back and lighten them, but that will still add visual clutter. Instead, let's add zebra striping: alternating rows are a different color. This will help us follow a

[63] http://localhost:8080/09-dashboard-design/table/draft/

row from the left to the right.

Day	Summary	Max Temp	Max Temp Time	Wind Speed	Precipitation	UV Index
2017-01-03	Clear throughout the day.	45.34	1483423200	7.63	rain	1
2017-01-04	Foggy in the evening.	46.96	1483509600	8.92	rain	2
2017-01-05	Mostly cloudy until evening.	43.53	1483628400	13.42	rain	2
2017-01-06	Mostly cloudy starting in the afternoon, continuing until evening.	47.13	1483682400	3.4	rain	1
2017-01-07	Overcast until evening.	45.5	1483736400	6.7	rain	2
2017-01-08	Overcast until evening.	42.47	1483812000	13.07	rain	2
2017-01-09	Partly cloudy until afternoon.	39.22	1483909200	14.66	rain	2
2017-01-10	Mostly cloudy until evening.	35.61	1484006400	11.76	snow	2
2017-01-11	Partly cloudy in the morning.	38.83	1484125200	6.46	rain	2
2017-01-12	Mostly cloudy until evening.	37.95	1484168400	6.19	rain	2

Our table, with zebra striping

Much better! Now we can focus on the values. There are a few good rules of thumb for text alignment in tables:

1. Align text to the left. This helps users scan the table, since we're used to reading left to right.
2. Align numerical values to the right. This helps with comparing different numbers within the column because their decimal points will line up.
3. Align the column headers with their values. This will help with scanning and keep the label close to the metric value.

Day	Summary	Max Temp	Max Temp Time	Wind Speed	Did Snow	UV Index
2017-01-03	Clear throughout the day.	45.34	1483423200	7.63	rain	1
2017-01-04	Foggy in the evening.	46.96	1483509600	8.92	rain	2
2017-01-05	Mostly cloudy until evening.	43.53	1483628400	13.42	rain	2
2017-01-06	Mostly cloudy starting in the afternoon, continuing until evening.	47.13	1483682400	3.4	rain	1
2017-01-07	Overcast until evening.	45.5	1483736400	6.7	rain	2
2017-01-08	Overcast until evening.	42.47	1483812000	13.07	rain	2
2017-01-09	Partly cloudy until afternoon.	39.22	1483909200	14.66	rain	2
2017-01-10	Mostly cloudy until evening.	35.61	1484006400	11.76	snow	2
2017-01-11	Partly cloudy in the morning.	38.83	1484125200	6.46	rain	2
2017-01-12	Mostly cloudy until evening.	37.95	1484168400	6.19	rain	2

Our table, aligned

Our numbers are still hard to compare, though, since they have different amounts of decimal points. Let's use `d3.format()` to ensure that all of our numbers are the same granularity. The number of decimal points we want will depend on the metric — note that we keep one decimal point for temperature but two decimal points for wind speed.

Let's also format our dates to make them more human friendly and get rid of redundant information.

Day	Summary	Max Temp	Max Temp Time	Wind Speed	Precipitation	UV Index
1/03	Clear throughout the day.	45.3	1 AM	7.63	rain	1
1/04	Foggy in the evening.	47.0	1 AM	8.92	rain	2
1/05	Mostly cloudy until evening.	43.5	10 AM	13.42	rain	2
1/06	Mostly cloudy starting in the afternoon, continuing until evening.	47.1	1 AM	3.40	rain	1
1/07	Overcast until evening.	45.5	4 PM	6.70	rain	2
1/08	Overcast until evening.	42.5	1 PM	13.07	rain	2
1/09	Partly cloudy until afternoon.	39.2	4 PM	14.66	rain	2
1/10	Mostly cloudy until evening.	35.6	7 PM	11.76	snow	2
1/11	Partly cloudy in the morning.	38.8	4 AM	6.46	rain	2
1/12	Mostly cloudy until evening.	38.0	4 PM	6.19	rain	2

Our table, formatted

These numbers are *still* hard to compare! The value `47.1` looks smaller than the next

value, `45.5`, because it takes up less space.

In most fonts, numbers have different widths to make them flow together. The font we're using, Inter UI, has a CSS property that makes each character equal-width:

```
font-feature-settings: 'tnum' 1;
```

If the font you're using doesn't have this feature, switch to a monospace font for your numbers.

Day	Summary	Max Temp	Max Temp Time	Wind Speed	Precipitation	UV Index
1/03	Clear throughout the day.	45.3	1 AM	7.63	rain	1
1/04	Foggy in the evening.	47.0	1 AM	8.92	rain	2
1/05	Mostly cloudy until evening.	43.5	10 AM	13.42	rain	2
1/06	Mostly cloudy starting in the afternoon, continuing until evening.	47.1	1 AM	3.40	rain	1
1/07	Overcast until evening.	45.5	4 PM	6.70	rain	2
1/08	Overcast until evening.	42.5	1 PM	13.07	rain	2
1/09	Partly cloudy until afternoon.	39.2	4 PM	14.66	rain	2
1/10	Mostly cloudy until evening.	35.6	7 PM	11.76	snow	2
1/11	Partly cloudy in the morning.	38.8	4 AM	6.46	rain	2
1/12	Mostly cloudy until evening.	38.0	4 PM	6.19	rain	2

Our table, with fixed-width numbers

This table is much more readable than it was, but we can do more to help users process it faster. Remember the dimensions in which we can represent data from **Chapter 7**? Those dimensions aren't restricted to charts. Let's create a color scale for **Max Temp** and **Wind Speed** and fill the background of each cell with its value's color.

We'll make sure to use semantic colors for quicker association: blue to red for temperature, and white to slate gray for wind speed. Additionally, having two different color scales helps keep these columns visually distinct.

Day	Summary	Max Temp	Max Temp Time	Wind Speed	Precipitation	UV Index
1/03	Clear throughout the day.	45.3	1 AM	7.63	rain	1
1/04	Foggy in the evening.	47.0	1 AM	8.92	rain	2
1/05	Mostly cloudy until evening.	43.5	10 AM	13.42	rain	2
1/06	Mostly cloudy starting in the afternoon, continuing until evening.	47.1	1 AM	3.40	rain	1
1/07	Overcast until evening.	45.5	4 PM	6.70	rain	2
1/08	Overcast until evening.	42.5	1 PM	13.07	rain	2
1/09	Partly cloudy until afternoon.	39.2	4 PM	14.66	rain	2
1/10	Mostly cloudy until evening.	35.6	7 PM	11.76	snow	2
1/11	Partly cloudy in the morning.	38.8	4 AM	6.46	rain	2
1/12	Mostly cloudy until evening.	38.0	4 PM	6.19	rain	2

Our table, with background colors

Great! Now we can quickly see that the first few days were warm and them cooled down, and the days are windy in little spurts.

Another way to make our table more scannable is to convert **UV Index** into symbols. Since it is a **discrete** metric, we can display a number of symbols to represent the value.

There is another column that we can convert to symbols: **Precipitation**. But we also notice that it's a **binary** metric, and there are only two possible values. Let's change it into a **Did Snow** column and only display a symbol if the value is `snow`.

Day	Summary	Max Temp	Max Temp Time	Wind Speed	Did Snow	UV Index
1/03	Clear throughout the day.	45.3	1 AM	7.63		✳
1/04	Foggy in the evening.	47.0	1 AM	8.92		✳✳
1/05	Mostly cloudy until evening.	43.5	10 AM	13.42		✳✳
1/06	Mostly cloudy starting in the afternoon, continuing until evening.	47.1	1 AM	3.40		✳
1/07	Overcast until evening.	45.5	4 PM	6.70		✳✳
1/08	Overcast until evening.	42.5	1 PM	13.07		✳✳
1/09	Partly cloudy until afternoon.	39.2	4 PM	14.66		✳✳
1/10	Mostly cloudy until evening.	35.6	7 PM	11.76	❄	✳✳
1/11	Partly cloudy in the morning.	38.8	4 AM	6.46		✳✳
1/12	Mostly cloudy until evening.	38.0	4 PM	6.19		✳✳

Our table, with symbols

Let's see if we can improve on **Max Temp Time**. Since the times span both `AM` and `PM`, they are hard to compare quickly. Instead, we can use another data dimension: **position**. Instead, let's display a line that is further right if the max temp occured later in the day.

Day	Summary	Max Temp	Max Temp Time	Wind Speed	Did Snow	UV Index
1/03	Clear throughout the day.	45.3	|	7.63		✳
1/04	Foggy in the evening.	47.0	|	8.92		✳✳
1/05	Mostly cloudy until evening.	43.5	|	13.42		✳✳
1/06	Mostly cloudy starting in the afternoon, continuing until evening.	47.1	|	3.40		✳
1/07	Overcast until evening.	45.5	|	6.70		✳✳
1/08	Overcast until evening.	42.5	|	13.07		✳✳
1/09	Partly cloudy until afternoon.	39.2	|	14.66		✳✳
1/10	Mostly cloudy until evening.	35.6	|	11.76	❄	✳✳
1/11	Partly cloudy in the morning.	38.8	|	6.46		✳✳
1/12	Mostly cloudy until evening.	38.0	|	6.19		✳✳

Our table, with time lines

This is much easier to scan! We can quickly see that the max temp was earlier in the day for the first four rows, then in the afternoon for the next four rows. As usual, how

you represent this metric will depend on your goals. If the specific time is important, maybe adding another column with the exact value would be helpful.

The last column we want to update is our **Summary** metric. The strings can be quite long and take up lots of space. Let's bump that down a font size to save space and decrease its visual importance.

Day	Summary	Max Temp	Max Temp Time	Wind Speed	Did Snow	UV Index
1/03	Clear throughout the day.	45.3	|	7.63		✳
1/04	Foggy in the evening.	47.0	|	8.92		✳✳
1/05	Mostly cloudy until evening.	43.5	|	13.42		✳✳
1/06	Mostly cloudy starting in the afternoon, continuing until evening.	47.1	|	3.40		✳
1/07	Overcast until evening.	45.5	|	6.70		✳✳
1/08	Overcast until evening.	42.5	|	13.07		✳✳
1/09	Partly cloudy until afternoon.	39.2	|	14.66		✳✳
1/10	Mostly cloudy until evening.	35.6	|	11.76	❄	✳✳
1/11	Partly cloudy in the morning.	38.8	|	6.46		✳✳
1/12	Mostly cloudy until evening.	38.0	|	6.19		✳✳

Our finished table

If we have a long table, we can remind our users what each column is for by sticking the header to the top. Thankfully, modern css makes this simple, we only need these lines of CSS:

```
thead th {
    position: sticky;
    top: 0;
}
```

To make the stickied header stand out while it's stuck, let's give it a dark background color and light text.

Additionally, let's highlight a row when the user hovers over it. This will allow them to easily compare values all the way across a row.

Day	Summary	Max Temp	Max Temp Time	Wind Speed	Did Snow	UV Index
2/13	Partly cloudy in the morning.	42.1		5.93		✱✱✱
2/14	Partly cloudy until evening.	39.8		2.24		✱✱✱
2/15	Windy starting in the evening.	44.4		20.56		✱✱✱
2/16	Mostly cloudy until evening.	60.8		12.02		✱✱✱
2/17	Windy in the morning and mostly cloudy until evening.	34.5		22.87	❄	✱✱✱
2/18	Partly cloudy until afternoon.	41.5		9.36		✱✱✱
2/19	Mostly cloudy until evening and windy until afternoon.	44.3		15.44		✱✱✱
2/20	Windy until afternoon and mostly cloudy until evening.	35.3		28.70	❄	✱✱✱
2/21	Mostly cloudy until evening.	36.1		11.55		✱✱✱✱
2/22	Overcast until evening.	35.8		14.22		✱✱✱

Table header sticks on scroll

Wonderful! If you haven't been following along, make sure to check out the final implementation at `/09-dashboard-design/table/completed/`.

Even though creating a table may not sound exciting, there is room to use our new knowledge of data types and data dimensions to make it more effective and easier to read.

Designing a dashboard layout

We've talked about how to build and design parts of a dashboard, but how do we pull it all together?

Let's look at a few tips for designing the dashboard's layout:

Have a main focus

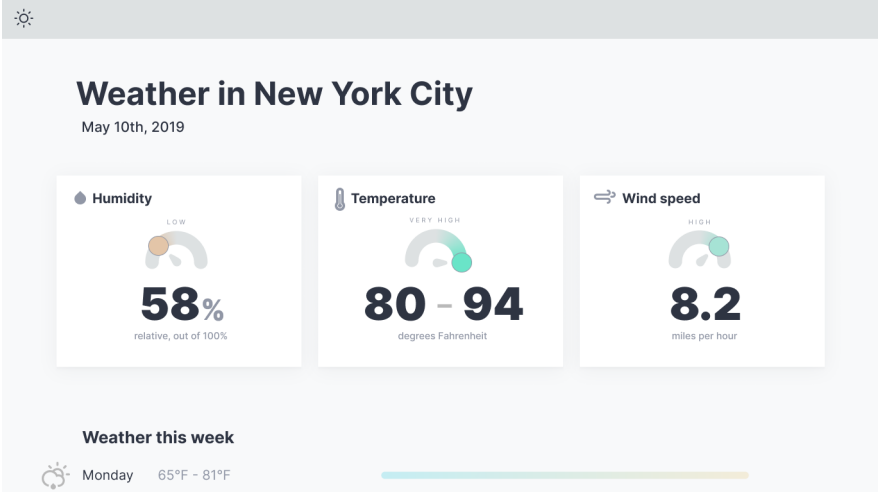
Many designs of dashboards throw all of the data on the page at once. While this might work in some cases, users can be easily overwhelmed. Aim for one main chart on screen at a time.

For example, if the main goal is to show how a metric changed over time, or to look for temporal trends, lead with a large timeline that the user can focus on.

Dashboard with main chart

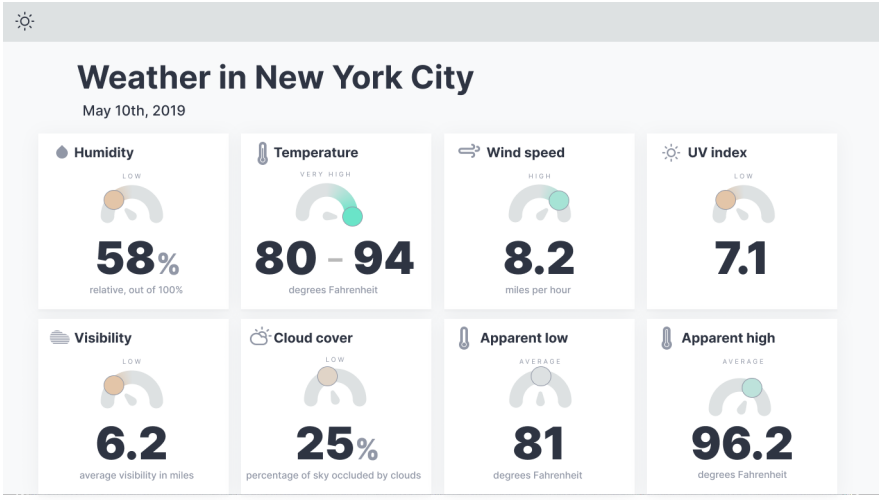
Keep the number of visible metrics small

If there isn't *one* main takeaway for users, showing multiple metrics can help to quickly convey an idea of the bigger picture. Make sure to give as much context as possible – how do these numbers compare to historical averages, or similar metrics?



Dashboard with three main metrics

Keep the number of metrics small though, to prevent making the user process a lot of information at once. Here's the same dashboard with eight metrics shown at once – a user could easily feel overwhelmed with this design. What should they look at first? When everything is important, nothing is.



Dashboard with eight main metrics

Data density can increase with a user's comfort – maybe add an option for a denser layout for power users?

Let the user dig in

We want to reduce the amount of information we show at once, but we still want to give our users the information they need. Once you're showing only what's most important, let the user **dig in** to a piece of information. The interaction will depend on the amount of extra information you want to show. If could involve any progressive disclosure technique, the main ones being:

A tooltip

to show an explanation of a metric, or 1-2 sentences of extra information.

A modal

to show a longer explanation that might include a detailed chart.

Expand in place

to show a longer explanation and/or a chart. This is a good option when the user wants to compare multiple items.

Switch to a detail page

to show much more information. A whole page gives enough space for multiple detailed charts and paragraphs of information. They also feel more concrete and allow users to bookmark or share the url.

Deciding questions

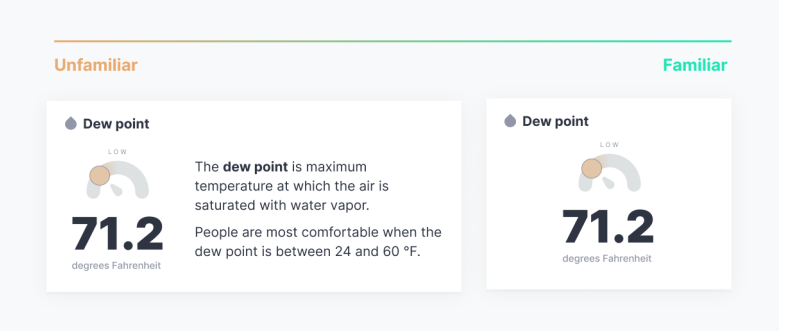
There isn't one layout that will fit any purpose, but a few choice questions can guide you towards a more suitable layout.

How familiar are users with the data?

For a dashboard that is also introducing users to the dataset, take the time to explain the nuances of the data. Make sure the answers to these questions are clear:

- where is the data from?
- how was the data collected?
- what are non-obvious nuances that might skew the data?
- what do these words mean? It's easy to use jargon that makes sense to *you*, but not to uninitiated users.

For example, the layout on the left might be more accessible for unfamiliar users, but might be totally unnecessary for a domain expert.

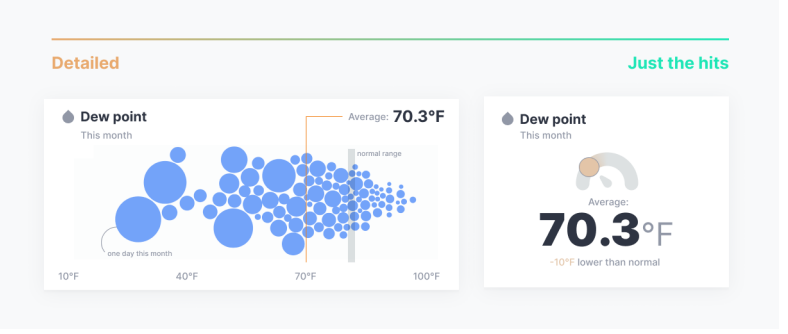


Metric layouts depending on user familiarity

How much time do users have to spend?

If your users are CEOs or people without much time on their hands, you want to give them the takeaways, and fast. What insights are most important to your users? Lead with those metrics, and save the secondary information for further down the screen, or a detail page.

For example, a user with more time on their hands might appreciate the extra detail from the left layout. They might even learn something extra! But for users in a hurry, the layout on the right gets to the point quickly and gives them only as much information as they need.



Metric layouts depending on how much time a user has

There are many ways to build a dashboard – hopefully this information will help on your next project. Remember to choose a main focus for each screen, give metrics context, and start with the goals of the dashboard.

Complex Data Visualizations

In these bonus chapters, we'll combine everything we've learned into three complex data visualizations. These walkthroughs will not be as in depth as the previous chapters — instead, they are meant to show you how to build on your new knowledge.

We'll take a higher-level look at each chart — what are the motivations, and what are the tricky code bits that crop up when making them. When you take your new skills and set out to make your own complicated, awesome data visualizations, you'll run into new challenges that we haven't covered here.

But fear not! You'll be able to use your solid foundation that we've built in this book to tackle those new puzzles. Let's get a sneak peak of how to approach these puzzles in the road ahead.

Marginal Histogram

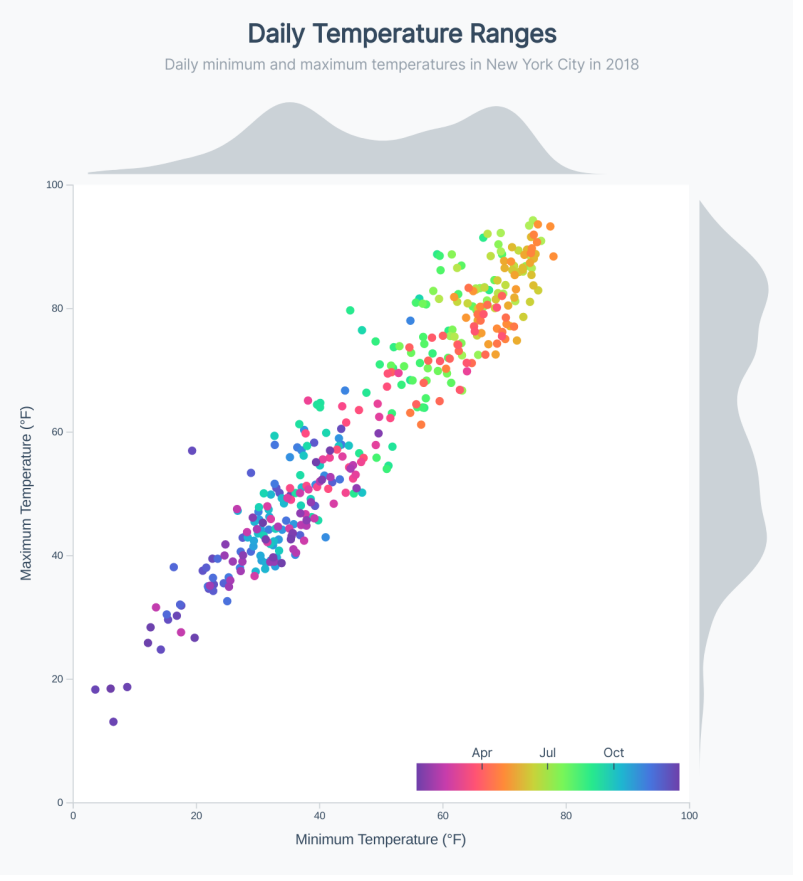
First, we'll focus on enhancing a chart we've already made: our scatter plot. This chart will have multiple goals, all exploring the daily temperature ranges in our weather dataset:

- do most days have a similar range in temperatures? Or do colder days vary less than warmer days?
- did we have any anomalous days? Were both the minimum *and* maximum temperatures anomalous, or just one?
- how do the temperatures change throughout the year?

As you can see, more complicated charts have the ability to answer multiple questions — we just need to make sure that they're focused enough to answer them well.

To help answer these questions, we'll add two main components to our scatter plot:

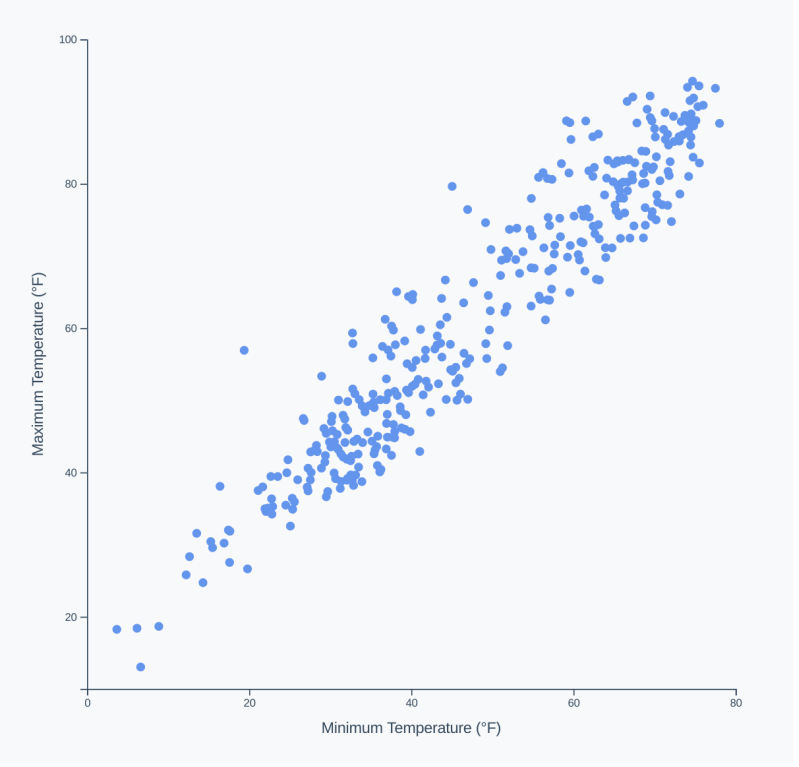
- a color scale that shows *when* the day falls in the year
- one histogram on the x axis to show the distribution of minimum temperatures and one histogram on the y axis to show the distribution of maximum temperatures



Marginal Histogram

To start, we can use the scatter plot code that we've already created. Let's start up our server in the terminal (`live-server`, if that's your preference) and navigate to the url http://localhost:8080/10-marginal-histogram/draft/[64]. We can see our old scatter plot friend, this time with the minimum daily temperature on the x axis and maximum daily temperature on the y axis.

[64] http://localhost:8080/10-marginal-histogram/draft/



Scatter plot

As usual, this code lives in the `/code/10-marginal-histogram/draft/` folder. You have two options in this chapter:

1. Follow along with the code examples and try to fill the missing code gaps. We won't go over every line of code in this chapter, but the completed code is available at `/code/10-marginal-histogram/completed/` if you need a reference.
2. Read through each example to follow the journey, then dive into the code and try to re-create the chart, using the `/completed/` folder as a reference.

The choice is yours, but make sure to choose whichever option will increase your confidence in your skills so you can launch into your own crazy custom data visualizations after.

Let's go through each of the enhancements of the final chart.

Chart bounds background

Let's start with an easy one — we can see that the final chart **bounds** have a white background. This lighter background serves two purposes:

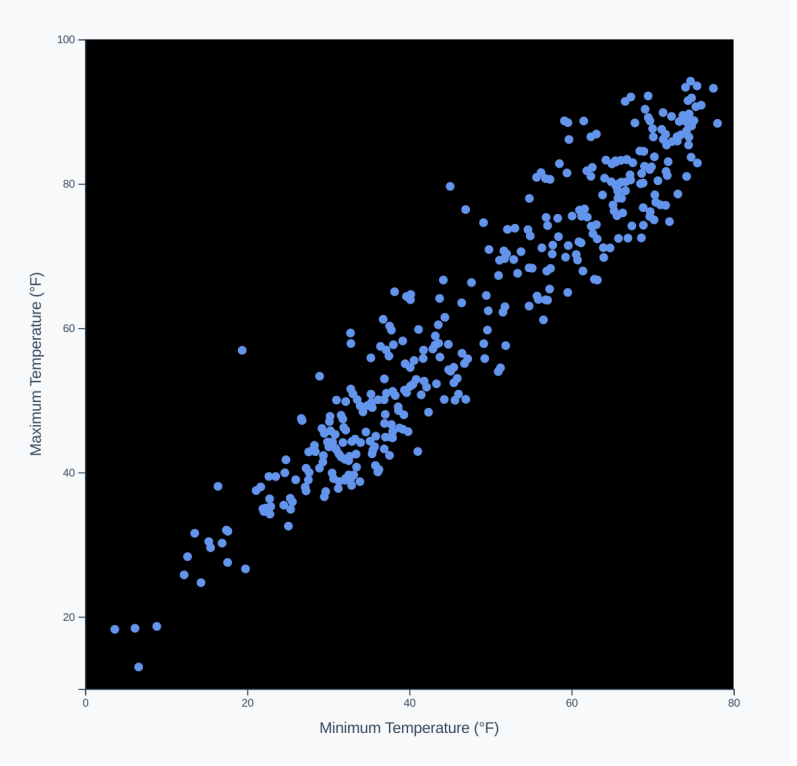
1. to help the reader focus on the chart, and
2. to clarify the edges of the x and y axis range.

Since our bounds are rectangular, this will be as simple as creating a new `<rect>` element with a `white` fill. Our main concern here is to draw our `<rect>` early on in our chart code to make sure it doesn't cover any other chart elements.

First, we'll create the `<rect>`:

code/10-marginal-histogram/completed/scatter.js

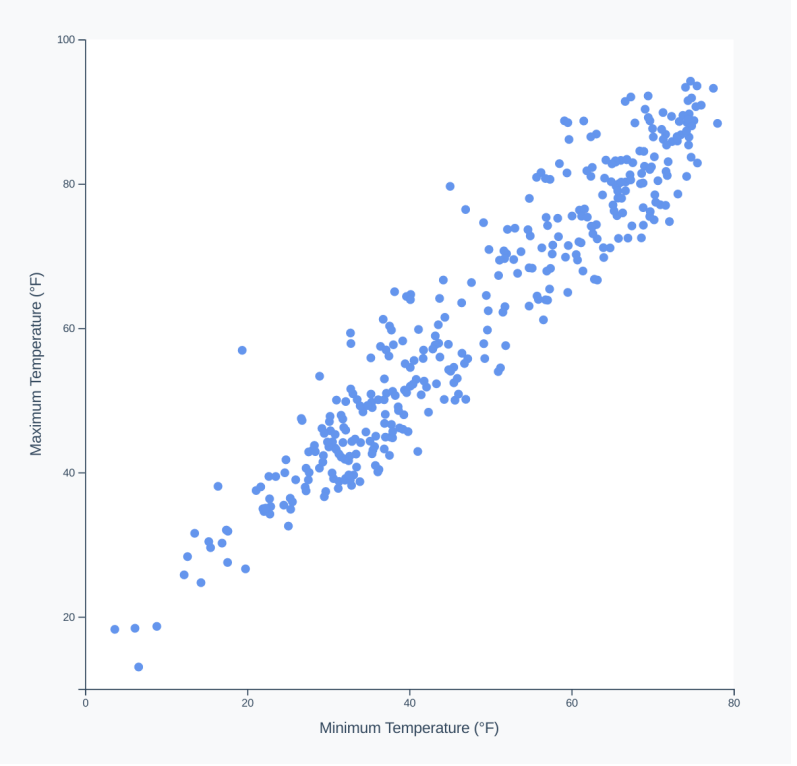
```
55  const boundsBackground = bounds.append("rect")
56      .attr("class", "bounds-background")
57      .attr("x", 0)
58      .attr("width", dimensions.boundedWidth)
59      .attr("y", 0)
60      .attr("height", dimensions.boundedHeight)
```



Scatter plot with bounds background

We'll set the `<rect>`'s fill in our CSS file, to keep our static styles out of the way of the general chart logic.

```
.bounds-background {
    fill: white;
}
```

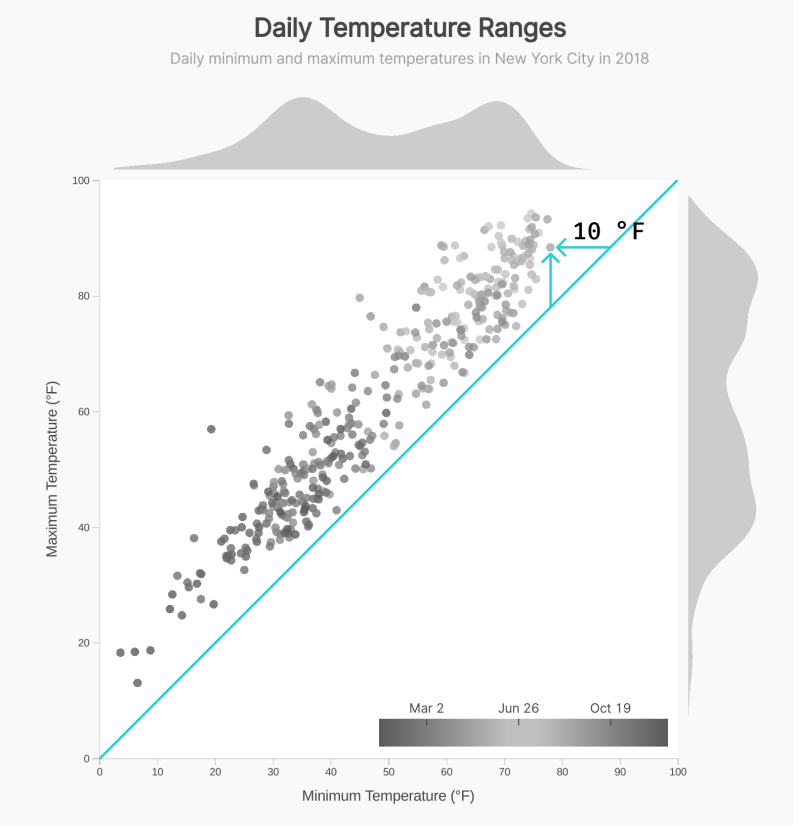


Scatter plot with white bounds background

Equal domains

Next, we'll notice that in our completed chart, our x and y scales have the same **domain** — they both run from `0` to `100` degrees (at least in the New York City data). This is to keep the frame of reference the same on both axes, so readers can more easily compare minimum and maximum temperatures.

If we draw a diagonal line from the bottom left of our bounds to the top right of our bounds, we can see the general range of temperatures in a day by looking at how far the dots are to the left (or to the top) of this line.



Scatter plot showing unity line distance

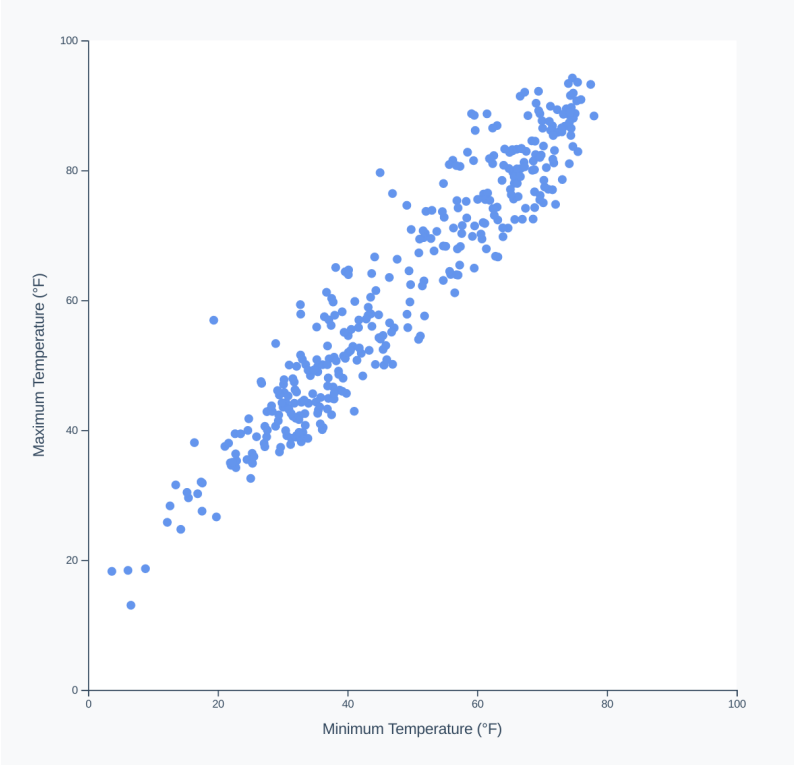
To find the smallest minimum temperature and the largest maximum temperature, we can find the extent of a new array of minimum and maximum values.

code/10-marginal-histogram/completed/scatter.js

```
64  const temperaturesExtent = d3.extent([
65    ...dataset.map(xAccessor),
66    ...dataset.map(yAccessor),
67  ])
```

Sometimes it's easier to hand **d3** a new array with multiple values per data point than to create several variables and choose the smallest and largest ones.

We can then use `temperaturesExtent` as the **domain** for both of our axes: `xScale` and `yScale`.



Scatter plot with same x and y domain

We could have used the same scale for both of our axes, since our `xScale` and `yScale` have the same **domain** and **range**, but it's also nice to keep them separate to be more explicit and to be prepared for future changes.

Color those dots

Next up, we'll make a more drastic change — we can see that the dots in our completed chart are colored based on their date.

We don't already have a way to access the date of a data point, so we'll scroll to the top of our `scatter.js` file and add a `colorAccessor`.

> To remind yourselves of the data structure, it's often a good idea to temporarily add a `console.table(dataset[0])` line right after we retrieve our dataset.

```
const parseDate = d3.timeParse("%Y-%m-%d")
const colorAccessor = d => parseDate(d.date)
```

Hmm, what if our dataset spans multiple years? We want to show *the time of year*, not the absolute date. Let's normalize our dates to all be in a specific year — that way we can color them according to their month and day, without taking the year into account.

code/10-marginal-histogram/completed/scatter.js

```
10  const colorScaleYear = 2000
11  const parseDate = d3.timeParse("%Y-%m-%d")
12  const colorAccessor = d => parseDate(d.date).setYear(colorScaleYear)
```

Great! Now we need to create a scale to convert our date objects into colors. We'll want to place this in our **Create Scales** section, probably after the other two, more basic, scales.

Let's use one of d3-scale-chromatic[65]'s built-in scales. `d3.interpolateRainbow()` will work here because our data is cyclical — we want our color scale to wrap around seamlessly. This will ensure that days in winter are similar colors, whether they're in December or January.

To create a scale that uses `d3.interpolateRainbow()`, we can use a sequential scale (`d3.scaleSequential()`) that covers all of the year 2000. Instead of setting a **range**, we'll use the `.interpolator()` method to use one of the built-in scales.

[65] https://github.com/d3/d3-scale-chromatic

```
const colorScale = d3.scaleSequential()
  .domain([
    d3.timeParse("%m/%d/%Y")(`1/1/${colorScaleYear}`),    // 1/1/2000
    d3.timeParse("%m/%d/%Y")(`12/31/${colorScaleYear}`), // 12/31/2000
  ])
  .interpolator(d3.interpolateRainbow)
```

> Note that we're setting our scale to cover dates in 2000. Even though our dataset has more recent dates, our `colorAccessor()` will modify them to preserve their month and day, but change their year to 2000, so that they are within our `colorScale`'s range.

This scale definitely works, but remember that we want our colors to have semantic meaning if possible.

d3.interpolateRainbow normal

The purple/blue colors in Winter work well, but burnt orange colors are often associated with Fall, and we have them representing Spring. Instead, let's invert our color scale so that orange-y colors represent Fall and green-y colors represent Spring.

d3.interpolateRainbow normal and inverted

Remember that `d3.interpolateRainbow()` is simply a function that turns a number into a color. We can invert it by flipping the sign of that number.

code/10-marginal-histogram/completed/scatter.js

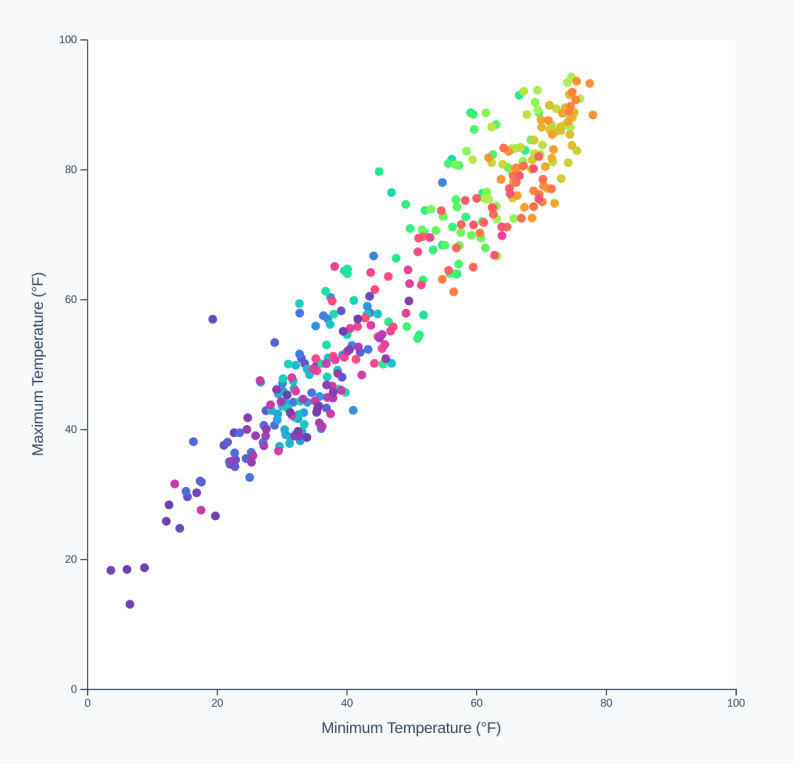
```
78  const colorScale = d3.scaleSequential()
79    .domain([
80      d3.timeParse("%m/%d/%Y")(`1/1/${colorScaleYear}`),
81      d3.timeParse("%m/%d/%Y")(`12/31/${colorScaleYear}`),
82    ])
83    .interpolator(d => d3.interpolateRainbow(-d))
```

Now that we have our color scale, we'll use it to color our dots. We'll need to find where we create our dots and set their attributes, then set their `fill` attribute.

code/10-marginal-histogram/completed/scatter.js

```
88  const dots = dotsGroup.selectAll(".dot")
89    .data(dataset, d => d[0])
90    .join("circle")
91      .attr("class", "dot")
92      .attr("cx", d => xScale(xAccessor(d)))
93      .attr("cy", d => yScale(yAccessor(d)))
94      .attr("r", 4)
95      .style("fill", d => colorScale(colorAccessor(d)))
```

Wonderful! Now our dots are all the colors of the rainbow:



Scatter plot with color

Without a legend or interactions, it's hard to tell what each color means (an important thing to remember when designing charts). However, we can see that the purple-y dots are near the bottom left, so we can guess that those are around January, when Winter hits New York.

Mini histograms

Next up, we'll tackle one of the bigger changes — adding a histogram to the top and right of the chart. These histograms will be great for giving readers a sense of how the minimum and maximum daily temperatures are distributed — some climates have a normal distribution for min temperatures, but a bimodal distribution for max temperatures!

We'll set the dimensions of our histograms (height and margin) in our `dimensions` object in the **Create chart dimensions** step to keep the chart size settings in

one place. This way, we know where to change things if we want to update the proportions of our chart.

We'll also want to bump up the `top` and `right` margins to account for our `histogramHeight` and `histogramMargin`.

```
let dimensions = {
 width: width,
 height: width,
 margin: {
   top: 90,
   right: 90,
   bottom: 50,
   left: 50,
 },
 histogramMargin: 10,
 histogramHeight: 70,
}
```

Let's start with the top histogram - first we want to generate the bins using `d3.histogram()`, similar to how we did in **Chapter 3**.

code/10-marginal-histogram/completed/scatter.js

```
97   const topHistogramGenerator = d3.histogram()
98     .domain(xScale.domain())
99     .value(xAccessor)
100    .thresholds(20)
101  // play around with the number of thresholds
102
103  const topHistogramBins = topHistogramGenerator(dataset)
```

Next, we'll use these bins to create a y scale for our histogram.

Note that we're creating a scale outside of our **Create scales** step. Now that our charts are getting more complicated, we get to take our knowledge and decide when to break the rules. If we want to strictly follow our chart checklist, we could move

these steps up into our **Create scales** step. We'll keep them in the **Draw data** step here, though, to group them with the following histogram drawing code. When you create your own charts, of course, the code organization will be totally up to you!

code/10-marginal-histogram/completed/scatter.js

```
105  const topHistogramYScale = d3.scaleLinear()
106    .domain(d3.extent(topHistogramBins, d => d.length))
107    .range([dimensions.histogramHeight, 0])
```

Next, we'll create a new **bounds** group for our top histogram and position it above our regular **bounds**.

code/10-marginal-histogram/completed/scatter.js

```
109  const topHistogramBounds = bounds.append("g")
110    .attr("transform", `translate(0, ${
111      -dimensions.histogramHeight
112      - dimensions.histogramMargin
113    }px)`)
```

We want to draw a `<path>` within these bounds, but first we need to figure out how to create our `<path>`'s `d` attribute string. If you remember from **Chapter 1**, we used `d3.line()` to create a `d` string generator to draw our timeline. We'll do a similar thing here, but using `d3.area()`, which is basically the same, but uses `y0` and `y1` methods instead of `y` to draw the top and bottom of our `<path>`'s shape.

code/10-marginal-histogram/completed/scatter.js

```
115  const topHistogramLineGenerator = d3.area()
116    .x(d => xScale((d.x0 + d.x1) / 2))
117    .y0(dimensions.histogramHeight)
118    .y1(d => topHistogramYScale(d.length))
119    .curve(d3.curveBasis)
```

You might be wondering: why don't we just use `d3.line()` and give it a `fill` instead of a `stroke`? The reason we want to use `d3.area()` is because the path created by `d3.line()` has no concept of *the bottom of the chart*. This will mostly work here, since our histogram starts and ends at the bottom of its bounds:

Area creating with `d3.line()`

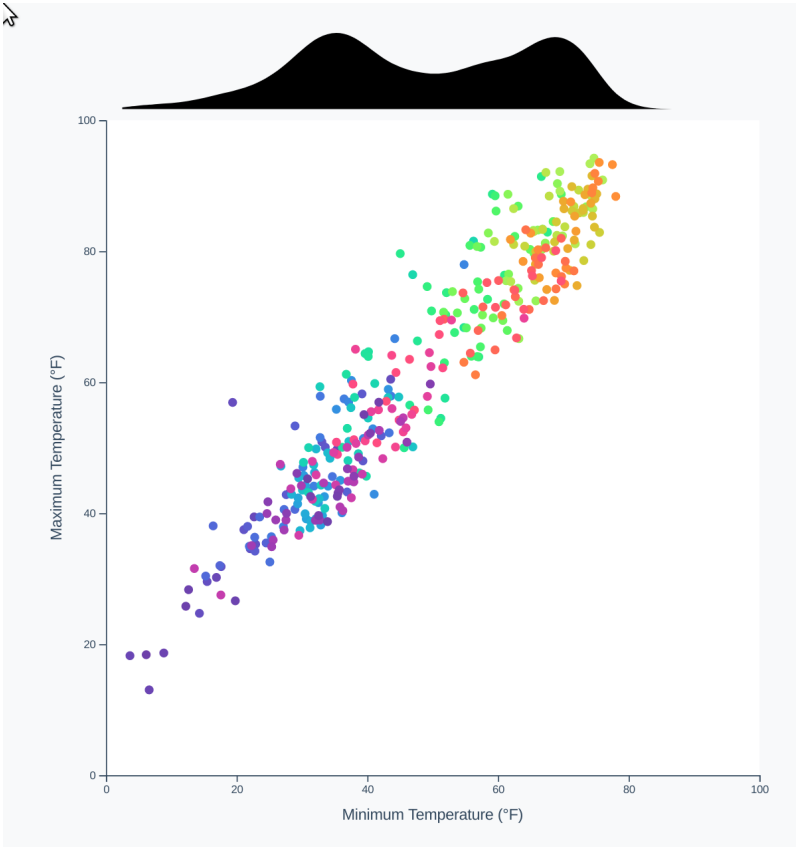
But if one side of our line ended in a different place, `d3.line()` would end the `d` string at that point, and the `<path>`'s fill would draw a point from the left-most point to the right-most point, slicing our area in half.

Area creating with `d3.line()`

Now we can finally use our `topHistogramLineGenerator()` to draw our histogram `<path>` element, hooking into CSS styles with a class attribute.

code/10-marginal-histogram/completed/scatter.js

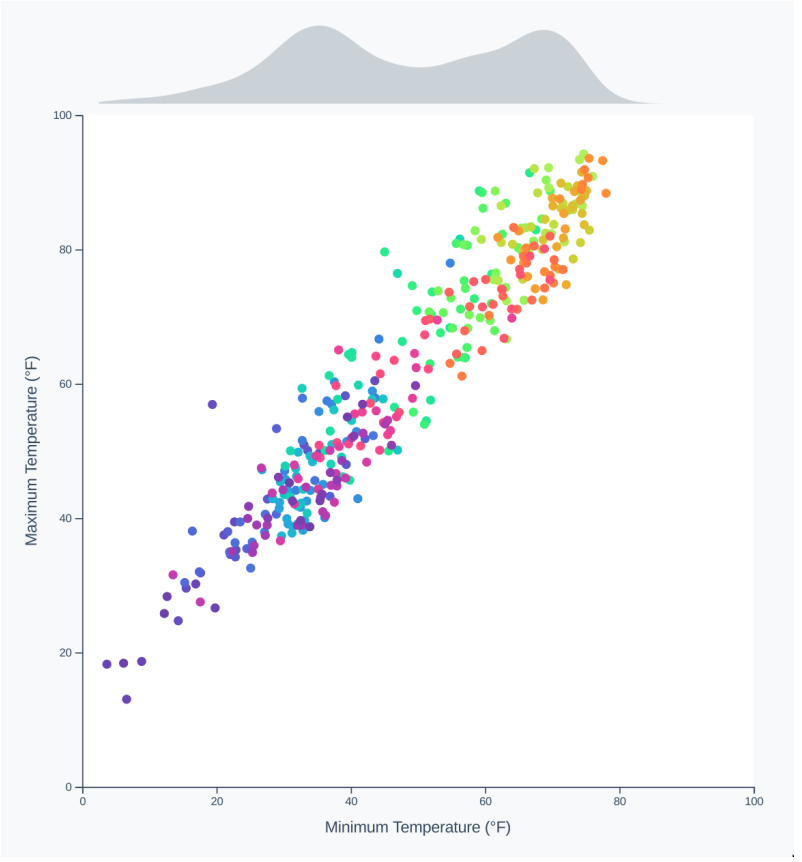
```
121   const topHistogramElement = topHistogramBounds.append("path")
122       .attr("d", d => topHistogramLineGenerator(topHistogramBins))
123       .attr("class", "histogram-area")
```



Scatter plot with top histogram

Great! Let's dim the color of our histogram so we don't overwhelm the rest of our chart.

```
.histogram-area {
    fill: #cbd2d7;
}
```



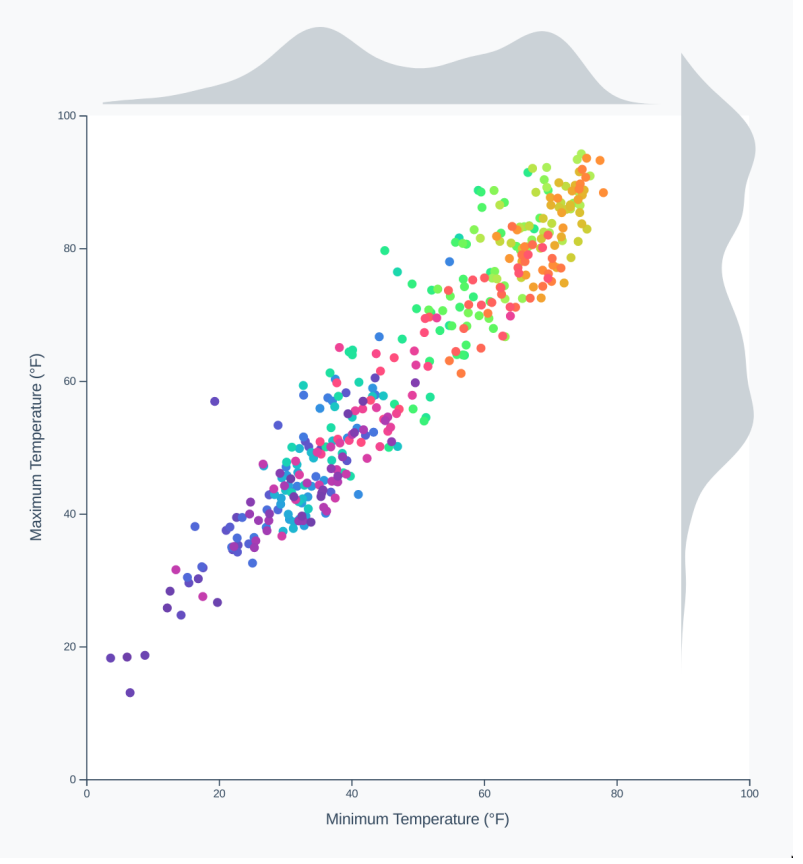
Scatter plot with top histogram, faded

Perfect! Now we can repeat these steps to draw a histogram on the right side of our chart.

code/10-marginal-histogram/completed/scatter.js

```
125   const rightHistogramGenerator = d3.histogram()
126     .domain(yScale.domain())
127     .value(yAccessor)
128     .thresholds(20)
129
130   const rightHistogramBins = rightHistogramGenerator(dataset)
131
132   const rightHistogramYScale = d3.scaleLinear()
133     .domain(d3.extent(rightHistogramBins, d => d.length))
134     .range([dimensions.histogramHeight, 0])
135
136   const rightHistogramBounds = bounds.append("g")
137       .attr("class", "right-histogram")
138       .style("transform", `translate(${
139         dimensions.boundedWidth + dimensions.histogramMargin
140       }px, -${
141         dimensions.histogramHeight
142       }px) rotate(90deg)`)
143
144   const rightHistogramLineGenerator = d3.area()
145     .x(d => yScale((d.x0 + d.x1) / 2))
146     .y0(dimensions.histogramHeight)
147     .y1(d => rightHistogramYScale(d.length))
148     .curve(d3.curveBasis)
149
150   const rightHistogramElement = rightHistogramBounds.append("path")
151       .attr("d", d => rightHistogramLineGenerator(rightHistogramBins))
152       .attr("class", "histogram-area")
```

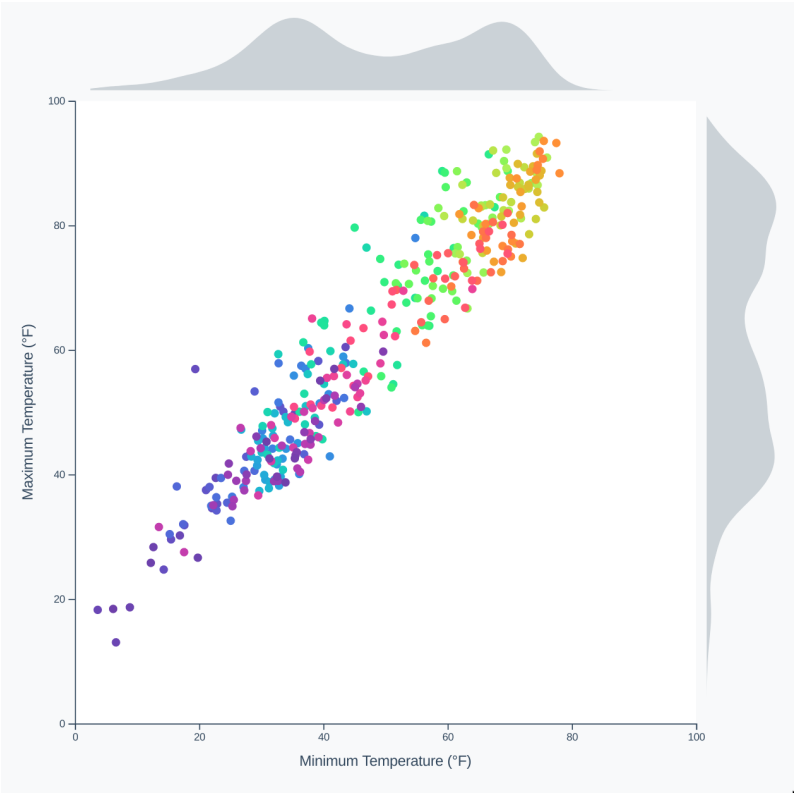
Notice that we're drawing our histogram vertically, similar to our top histogram, and then rotating it 90 degrees (clockwise).



Scatter plot with right histogram

Hmm, we've rotated our histogram around the wrong axis though. Don't worry! We can set the rotation axis with the `transform-origin` css property, setting it to the bottom, left of our histogram group (instead of the default top, left).

```
.right-histogram {
    transform-origin: 0 70px;
}
```

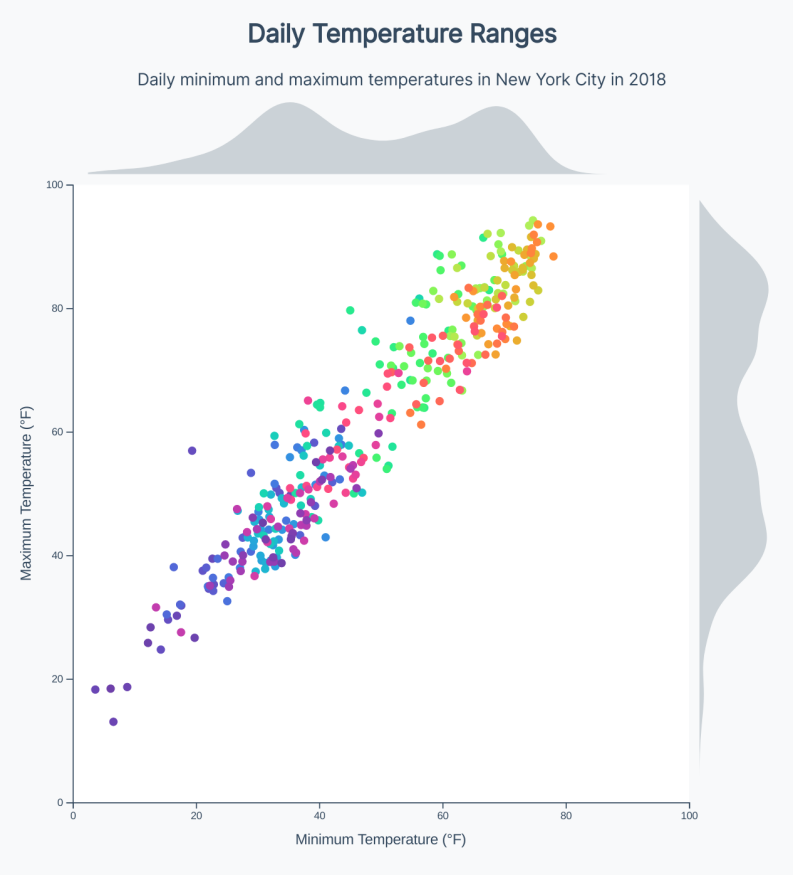


Scatter plot with right histogram, transformed correctly

Static polish

Before we turn our attention to the interactive features, let's do a little bit of clean-up. Switching over to our HTML file, let's add a title and description above our `#wrapper` element. You might want to use different text — what will describe the chart to readers as succinctly as possible, but still tell them what they need to know.

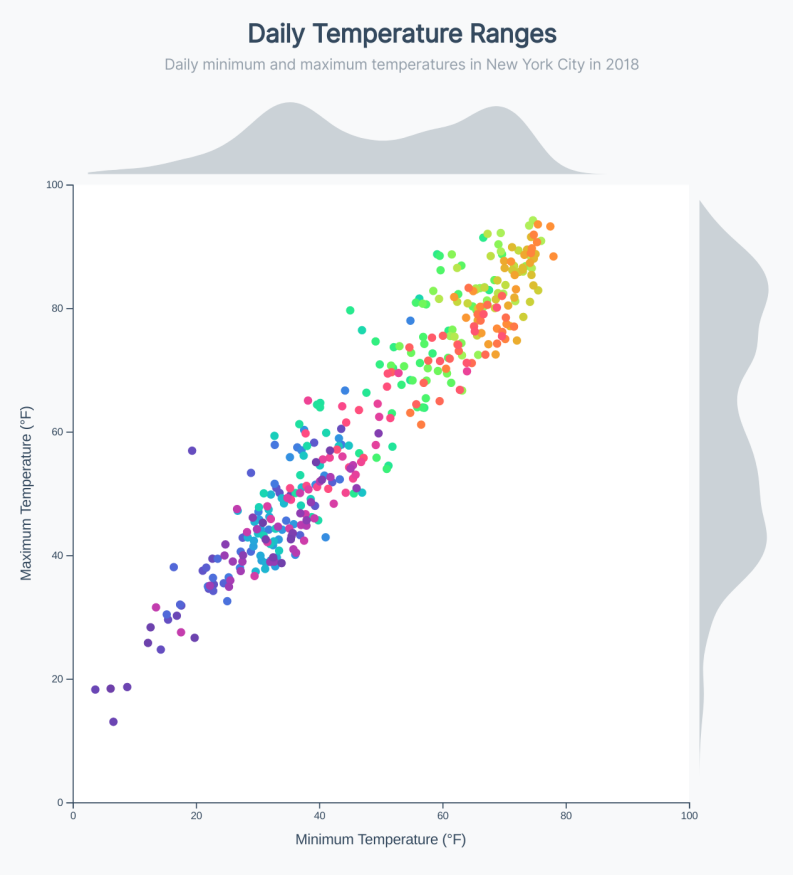
```
<h2 class="title">Daily Temperature Ranges</h2>
<div class="description">Daily minimum and maximum temperatures in New \
York City in 2018</div>
```



Scatter plot with title and description

Next, let's style our text. We have three options here:

- try to mimic the styles in the following screenshot, practicing your CSS skills,
- copy the styles from the `/completed/styles.css` file, or
- make up your own styles!

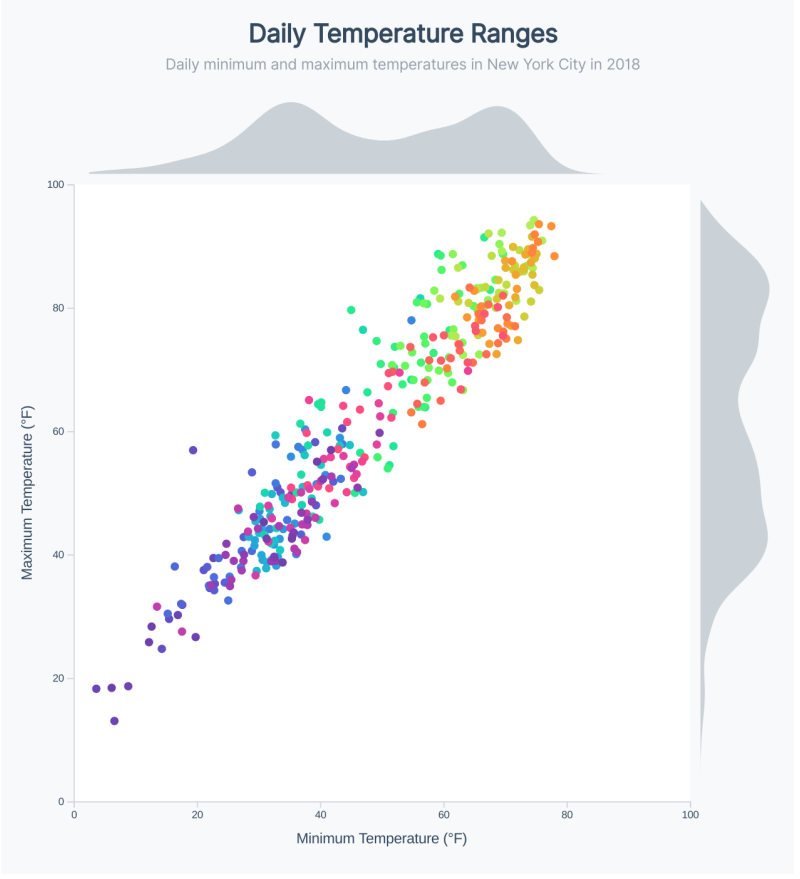


Scatter plot with styled title and description

Let's also dim the lines in our axes — the bounds of our chart aren't very important here, and we don't want to grab the reader's attention with them.

```
.tick line,
.domain {
    color: #cfd4d8;
}
```

This color was chosen because it's a lighter, desaturated version of the dark color we're using for our text. Staying in the same "family" of grey prevents our greys from clashing.



Scatter plot with lightened axis lines

Adding a tooltip

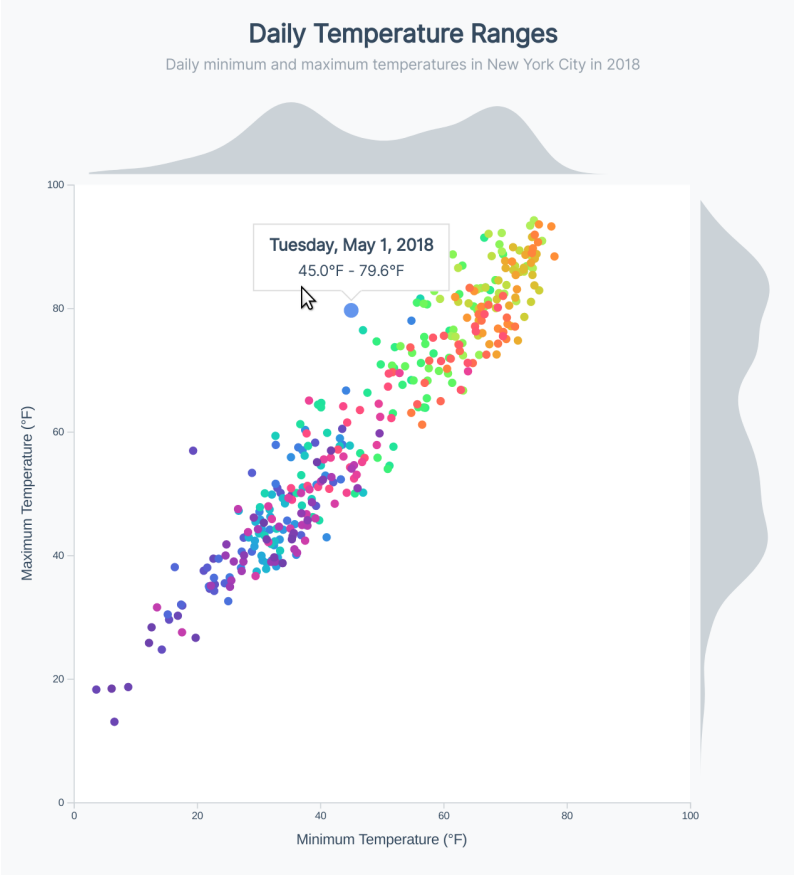
Let's move on to adding interactions! Let's copy what we did in **Chapter 5** with adding voronoi polygons for a scatter plot tooltip. We won't look at the code here,

but feel free to use our `code/05-interactions/3-scatter/draft/scatter.js` code as a reference.

This will require updating three files:

- `index.html` to add the tooltip and text elements,
- `scatter.js` to add the voronoi polygons and mouse events, and
- `styles.css` to style and control the tooltip opacity.

> We'll use the function names `onVoronoiMouseEnter()` and `onVoronoiMouseLeave()`, since we'll be adding other mouse events to our legend.

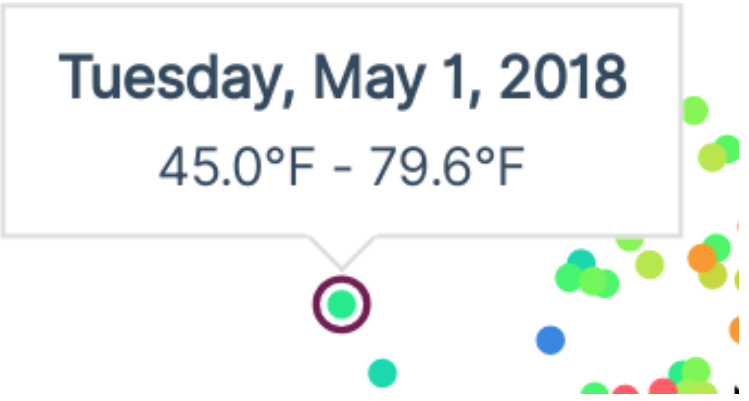


Scatter plot with tooltip

Alright! Let's make one improvement here — the color of the dot is meaningful, but we're hiding it with an overlapping dot on hover. Let's bump up the size of that hover dot and update the *styles* to have a stroke and no fill.

```
.tooltip-dot {
    fill: none;
    stroke: #6F1E51;
    stroke-width: 2px;
}
```

Now we can see the color of the dot we're hovering, while still showing which dot is hovered.



Scatter plot with tooltip with circle dot

Histogram hover effects

We're doing a great job with showing what *dot* a reader is hovered, but it's not clear where that dot's min and max temperature is on the respective histograms.

Let's create a `<g>` element to append our hover elements to — that way we only have to *show* and *hide* one element on mouse enter & leave.

code/10-marginal-histogram/completed/scatter.js

```
263   const hoverElementsGroup = bounds.append("g")
264       .attr("opacity", 0)
```

Make sure you also add `hoverElementsGroup.style("opacity", 1)` to the `onVoronoiMouseEnter()` function and `hoverElementsGroup.style("opacity", 0)` to the `onVoronoiMouseLeave()` function.

After we draw our voronoi polygons, let's add a horizontal and a vertical line that we can move in our hover event. We'll want to use `<rect>` elements, since `<line>`s' attributes (`x1`, `x2`,etc) won't respect CSS transitions, but `<path>`s' `d` attribute will. First, we'll create our elements just before our `onVoronoiMouseEnter()` function.

code/10-marginal-histogram/completed/scatter.js

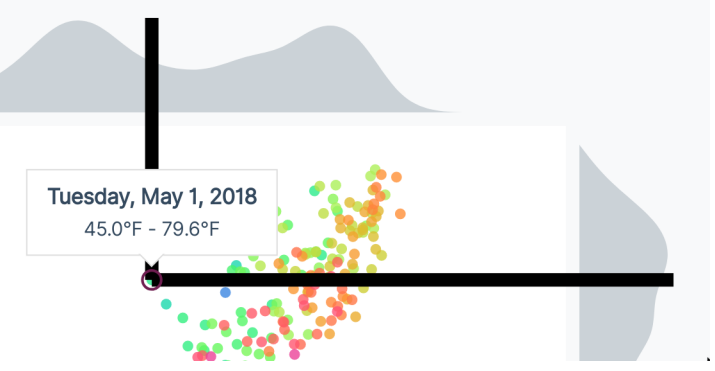
```
267   const horizontalLine = hoverElementsGroup.append("rect")
268       .attr("class", "hover-line")
269   const verticalLine = hoverElementsGroup.append("rect")
270       .attr("class", "hover-line")
```

Now we can update our `<rect>`s attributes at the bottom of our `onVoronoiMouseEnter()` function, after we define our hovered `x` and `y`.

code/10-marginal-histogram/completed/scatter.js

```
285     const hoverLineThickness = 10
286     horizontalLine.attr("x", x)
287       .attr("y", y - hoverLineThickness / 2)
288       .attr("width", dimensions.boundedWidth
289         + dimensions.histogramMargin
290         + dimensions.histogramHeight
291         - x)
292       .attr("height", hoverLineThickness)
293     verticalLine.attr("x", x - hoverLineThickness / 2)
294       .attr("y", -dimensions.histogramMargin
295         - dimensions.histogramHeight)
```

Now we can see our rectangles when we hover over dots!



Scatter plot with hover lines

Those black lines are a bit intense — let's change the color by setting the `fill` in our CSS file.

```
.hover-line {
    fill: #5758BB;
}
```

Now we can see our lines showing on hover!

Scatter plot with hover lines

You might notice that these lines flicker when hovering around the chart. This is due to this chain of events: